

1. Muestre la secuencia de reducciones que corresponde a la llamada:
map (fn s => s ^ s) ["uno", "dos", "tres"]

```
map f [ ] = [ ]  
map f (hd::tl) = (f hd) :: (map f tl)
```

- map f ["uno","dos","tres"]
- map f ("uno"::["dos","tres"])
=> (f "uno") :: (map f ["dos","tres"])
- f "uno"
= ("uno" ^ "uno")
= "unouno" 
- => "unouno" :: (map f ["dos","tres"])
- map f ["dos","tres"]
=> (f "dos") :: (map f ["tres"])
=> "dosdos" :: (map f ["tres"]) 
- map f ["tres"]
=> (f "tres") :: (map f [])
=> "trestres" :: (map f [])
- map f [] = [] 
- map f ["tres"] => "trestres" :: []
map f ["dos","tres"] => "dosdos" :: ("trestres" :: [])
map f ["uno","dos","tres"]
=> "unouno" :: ("dosdos" :: ("trestres" :: []))
=> ["unouno","dosdos","trestres"]
- val it = ["unouno","dosdos","trestres"] : string list 

2. Determine el tipo (más general) de la función map usando la inferencia de tipos de Hindley Milner

Suponiendo que $f : \alpha \rightarrow \beta$
la lista de entrada tiene elementos de tipo α , o sea α list

```
map f (hd::tl) = (f hd) :: (map f tl)
```

- De $hd::tl$ se deduce: 
- $hd : \alpha$
- $tl : \alpha$ list

Luego:

- $f \text{ hd} : \beta$ (porque $f : \alpha \rightarrow \beta$ y $\text{hd} : \alpha$)
- El operador $::$ construye listas: si $x : \beta$ y $xs : \beta \text{ list}$, entonces $x :: xs : \beta \text{ list}$.

Así que:

- $(f \text{ hd}) :: (\text{map } f \text{ tl})$ obliga a que $\text{map } f \text{ tl} : \beta \text{ list}$

Como $\text{tl} : \alpha \text{ list}$,
entonces $\text{map } f : \alpha \text{ list} \rightarrow \beta \text{ list}$

Por último, como map recibe primero f y luego una lista:

- $\text{map} : (\alpha \rightarrow \beta) \rightarrow \alpha \text{ list} \rightarrow \beta \text{ list}$

En notación de SML:

- $\text{map} : ('a \rightarrow 'b) \rightarrow 'a \text{ list} \rightarrow 'b \text{ list}$

3. Redacte un párrafo en el cual se detalle el razonamiento (proceso) para determinar el tipo

Para sacar el tipo de map con Hindley–Milner, lo que hacemos es tratar los tipos como incógnitas al inicio. Primero asumimos que f es una función que recibe algo de tipo $'a$ y devuelve algo de tipo $'b$. Luego miramos el patrón $(\text{hd}::\text{tl})$: eso nos dice que la lista de entrada no está vacía, que hd es un elemento de la lista y tl es el resto, así que ambos deben ser del mismo tipo: $\text{hd} : 'a$ y $\text{tl} : 'a \text{ list}$. Después observamos la parte derecha: $(f \text{ hd}) :: (\text{map } f \text{ tl})$. Como $f \text{ hd}$ devuelve un $'b$, entonces el primer elemento de la lista resultado es $'b$. Y como $::$ pega un elemento al frente de una lista, el otro lado $(\text{map } f \text{ tl})$ también tiene que ser una lista de $'b$, es decir, $\text{map } f \text{ tl} : 'b \text{ list}$. Con eso concluimos que map toma una función $'a \rightarrow 'b$ y una lista de $'a$, y produce una lista de $'b$. Por eso el tipo más general queda: $\text{map} : ('a \rightarrow 'b) \rightarrow 'a \text{ list} \rightarrow 'b \text{ list}$.